

AMQP is truly a messaging protocol in the sense that it does not by itself support, for example, publish-subscribe primitives. It expects that such issues are handled by more advanced, proprietary queue managers, akin to the message brokers discussed in Section 4.3.

Finally, there is no reason why a queue manager cannot be connected to another queue manager. In fact, it is quite common to organize queue managers into an overlay network in which messages are routed from producers to their consumers. AMQP does not specify how the overlay network should be constructed and managed, and, indeed, different providers of AMQP-based systems offer different solutions. Of particular importance is specifying how messages should be routed through the network. The bottom line is that administrators will need to do a lot of this specification manually. Only in cases where overlays have regular structures, such as cycles or trees, it becomes easier to provide the necessary routing details.

4.4 Multicast communication

An important topic in communication in distributed systems is the support for sending data to multiple receivers, also known as multicast communication. For many years, this topic has belonged to the domain of network protocols, where numerous proposals for network-level and transport-level solutions have been implemented and evaluated [Janic, 2005; Obraczka, 1998]. A major issue in all solutions was setting up the communication paths for information dissemination. In practice, this involved a huge management effort, in many cases requiring human intervention. In addition, as long as there is no convergence of proposals, ISPs have shown to be reluctant to support multicasting [Diot et al., 2000].

With the advent of peer-to-peer technology, and notably structured overlay management, it became easier to set up communication paths. As peer-to-peer solutions are typically deployed at the application layer, various application-level multicasting techniques have been introduced. In this section, we will take a brief look at these techniques.

Multicast communication can also be accomplished in other ways than setting up explicit communication paths. As we also explore in this section, gossip-based information dissemination provides simple (yet often less efficient) ways for multicasting.

Application-level tree-based multicasting

The basic idea in application-level multicasting is that nodes organize into an overlay network, which is then used to disseminate information to its members. An important observation is that network routers are not involved in group membership. As a consequence, the connections between nodes in the overlay network may cross several physical links, and as such, routing

messages within the overlay may not be optimal in comparison to what could have been achieved by network-level routing.

A crucial design issue is the construction of the overlay network. In essence, there are two approaches [El-Sayed et al., 2003; Hosseini et al., 2007; Allani et al., 2009]. First, nodes may organize themselves directly into a tree, meaning that there is a unique (overlay) path between every pair of nodes. An alternative approach is that nodes organize into a mesh network in which every node will have multiple neighbors and, in general, there exist multiple paths between every pair of nodes. The main difference between the two is that the latter generally provides higher robustness: if a connection breaks (e.g., because a node fails), there will still be an opportunity to disseminate information without having to immediately reorganize the entire overlay network.

Note 4.13 (Advanced: Constructing a multicast tree in Chord)

To make matters concrete, let us consider a relatively simple scheme for constructing a multicast tree in Chord, which we described in Note 2.5. This scheme was originally proposed for Scribe [Castro et al., 2002b] which is an application-level multicasting scheme built on top of Pastry [Rowstron and Druschel, 2001]. The latter is also a DHT-based peer-to-peer system.

Assume a node wants to start a multicast session. To this end, it simply generates a multicast identifier, say *mid* which is just a randomly-chosen 160-bit key. It then looks up *succ(mid)*, which is the node responsible for that key, and promotes it to become the root of the multicast tree that will be used to sending data to interested nodes. In order to join the tree, a node *P* simply executes the operation *lookup(mid)* having the effect that a lookup message with the request to join the multicast group *mid* will be routed from *P* to *succ(mid)*. The routing algorithm itself will be explained in detail in Chapter 5.

On its way toward the root, the join request will pass several nodes. Assume it first reaches node *Q*. If *Q* had never seen a join request for *mid* before, it will become a **forwarder** for that group. At that point, *P* will become a child of *Q* whereas the latter will continue to forward the join request to the root. If the next node on the root, say *R* is also not yet a forwarder, it will become one and record *Q* as its child and continue to send the join request.

On the other hand, if *Q* (or *R*) is already a forwarder for *mid*, it will also record the previous sender as its child (i.e., *P* or *Q*, respectively), but there will not be a need to send the join request to the root anymore, as *Q* (or *R*) will already be a member of the multicast tree.

Nodes such as *P* that have explicitly requested to join the multicast tree are, by definition, also forwarders. The result of this scheme is that we construct a multicast tree across the overlay network with two types of nodes: pure forwarders that act as helpers, and nodes that are also forwarders, but have explicitly requested to join the tree. Multicasting is now simple: a node merely sends a multicast message toward the root of the tree by again executing the *lookup(mid)* operation, after which that message can be sent along the tree.

We note that this high-level description of multicasting in Scribe does not do complete justice to its original design. The interested reader is encouraged to take a look at the details, which can be found in [Castro et al., 2002b].

Performance issues in overlays

From the high-level description given above, it should be clear that although building a tree by itself is not that difficult once we have organized the nodes into an overlay, building an efficient tree may be a different story. Note that in our description so far, the selection of nodes that participate in the tree does not take into account any performance metrics: it is purely based on the (logical) routing of messages through the overlay.

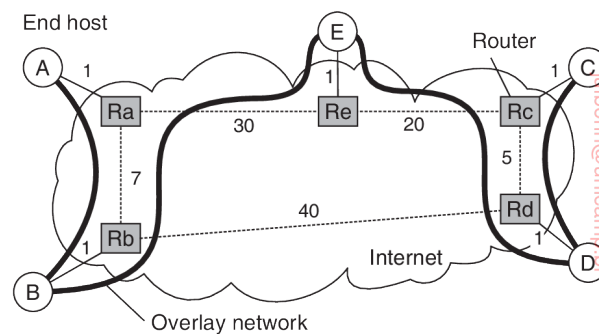


Figure 4.35: The relation between links in an overlay and actual network-level routes.

To understand the problem at hand, take a look at Figure 4.35 which shows a small set of five nodes that are organized in a simple overlay network, with node A forming the root of a multicast tree. The costs for traversing a physical link are also shown. Now, whenever A multicasts a message to the other nodes, it is seen that this message will traverse each of the links $\langle B, Rb \rangle$, $\langle Ra, Rb \rangle$, $\langle E, Re \rangle$, $\langle Rc, Rd \rangle$, and $\langle D, Rd \rangle$ twice. The overlay network would have been more efficient if we had not constructed *overlay links* $\langle B, E \rangle$ and $\langle D, E \rangle$, but instead $\langle A, E \rangle$ and $\langle C, E \rangle$. Such a configuration would have saved the double traversal across physical links $\langle Ra, Rb \rangle$ and $\langle Rc, Rd \rangle$.

The quality of an application-level multicast tree is generally measured by three different metrics: link stress, stretch, and tree cost. **Link stress** is defined per link and counts how often a packet crosses the same link [Chu et al., 2002]. A link stress greater than 1 comes from the fact that although at a logical level a packet may be forwarded along two different connections, part of those connections may actually correspond to the same physical link, as we showed in Figure 4.35.

The **stretch** or **relative delay penalty (RDP)** measures the ratio in the delay between two nodes in the overlay, and the delay that those two nodes would experience in the underlying network. For example, messages from B to C follow the route $B \rightarrow Rb \rightarrow Ra \rightarrow Re \rightarrow E \rightarrow Re \rightarrow Rc \rightarrow Rd \rightarrow D \rightarrow Rd \rightarrow Rc \rightarrow C$ in the overlay network, having a total cost of 73 units. However, messages would have been routed in the underlying network along the path $B \rightarrow Rb \rightarrow Rd \rightarrow Rc \rightarrow C$, with a total cost of 47 units, leading to a stretch of 1.55. Obviously, when constructing an overlay network, the goal is to minimize the aggregated stretch, or similarly, the average RDP measured over all node pairs.

Finally, the **tree cost** is a global metric, generally related to minimizing the aggregated link costs. For example, if the cost of a link is taken to be the delay between its two end nodes, then optimizing the tree cost boils down to finding a minimal spanning tree in which the total time for disseminating information to all nodes is minimal.

To simplify matters somewhat, assume that a multicast group has an associated and well-known node that keeps track of the nodes that have joined the tree. When a new node issues a join request, it contacts this **rendezvous node** to obtain a (potentially partial) list of members. The goal is to select the best member that can operate as the new node's parent in the tree. Who should it select? There are many alternatives and different proposals often follow very different solutions.

Consider, for example, a multicast group with only a single source. In this case, the selection of the best node is obvious: it should be the source (because in that case we can be assured that the stretch will be equal to 1). However, in doing so, we would introduce a star topology with the source in the middle. Although simple, it is not difficult to imagine the source may easily become overloaded. In other words, selection of a node will generally be constrained in such a way that only those nodes may be chosen who have k or less neighbors, with k being a design parameter. This constraint severely complicates the tree-establishment algorithm, as a good solution may require that part of the existing tree is reconfigured. Tan et al. [2003] provide an extensive overview and evaluation of various solutions to this problem.

Note 4.14 (Advanced: Switch-trees)

As an illustration, let us take a closer look at one specific family, known as **switch-trees** [Helder and Jamin, 2002]. The basic idea is simple. Assume we already have a multicast tree with a single source as root. In this tree, a node P can switch parents by dropping the link to its current parent in favor of a link to another node. The only constraints imposed on switching links is that the new parent can never be a member of the subtree rooted at P (as this would partition the tree and create a loop), and that the new parent will not have too many immediate children. This last requirement is needed to limit the load of forwarding messages

by any single node.

There are different criteria for deciding to switch parents. A simple one is to optimize the route to the source, effectively minimizing the delay when a message is to be multicast. To this end, each node regularly receives information on other nodes (we will explain one specific way of doing this below). At that point, the node can evaluate whether another node would be a better parent in terms of delay along the route to the source, and if so, initiates a switch.

Another criteria could be whether the delay to the potential other parent is lower than to the current parent. If every node takes this as a criterion, then the aggregated delays of the resulting tree should ideally be minimal. In other words, this is an example of optimizing the cost of the tree as we explained above. However, more information would be needed to construct such a tree, but as it turns out, this simple scheme is a reasonable heuristic leading to a good approximation of a minimal spanning tree.

As an example, consider the case where a node P receives information on the neighbors of its parent. Note that the neighbors consist of P's grandparent, along with the other siblings of P's parent. Node P can then evaluate the delays to each of these nodes and subsequently choose the one with the lowest delay, say Q, as its new parent. To that end, it sends a switch request to Q. To prevent loops from being formed due to concurrent switching requests, a node that has an outstanding switch request will simply refuse to process any incoming requests. In effect, this leads to a situation where only completely independent switches can be carried out simultaneously. Furthermore, P will provide Q with enough information to allow the latter to conclude that both nodes have the same parent, or that Q is the grandparent.

An important problem that we have not yet addressed is node failure. In the case of switch-trees, a simple solution is proposed: whenever a node notices that its parent has failed, it simply attaches itself to the root. At that point, the optimization protocol can proceed as usual and will eventually place the node at a good point in the multicast tree. Experiments described in [Helder and Jamin, 2002] show that the resulting tree is indeed close to a minimal spanning one.

Flooding-based multicasting

So far, we have assumed that when a message is to be multicast, it is to be received by every node in the overlay network. Strictly speaking, this corresponds to **broadcasting**. In general, multicasting refers to sending a message to a subset of all the nodes, that is, a specific **group of nodes**. A key design issue when it comes to multicasting is to minimize the use of intermediate nodes for which the message is not intended. To make this clear, if the overlay is organized as a multi-level tree, yet only the leaf nodes are the ones who should receive a multicast message, then clearly there may be quite some nodes who need to store and subsequently forward a message that is not meant for them.

One simple way to avoid such inefficiency, is to construct an overlay network *per multicast group*. As a consequence, multicasting a message m to a group G is the same as broadcasting m to G . The drawback of this solution is that a node belonging to several groups, will, in principle, need to maintain a separate list of its neighbors for each group of which it is a member.

If we assume that an overlay corresponds to a multicast group, and thus that we need to broadcast a message, a naive way of doing so is to apply **flooding**. In this case, each node simply forwards a message m to each of its neighbors, except to the one from which it received m . Furthermore, if a node keeps track of the messages it received and forwarded, it can simply ignore duplicates. We will roughly see twice as many messages being sent as there are links in the overlay network, making flooding quite inefficient.

To understand the performance of flooding, we model an overlay network as a connected graph G with N nodes and M edges. Keep in mind that flooding means that we need to send (at least) M messages. Only if G is a tree, will flooding be optimal, for in that case, $M = N - 1$. In the worst case, when G is fully connected, we will have to send out $M = \binom{N}{2} = \frac{1}{2} \cdot N \cdot (N - 1)$ messages.

Suppose now that we have no information on the structure of the overlay network and that the best we can assume is that it can be represented as a **random graph**, which (when keeping matters simple) is a graph having a probability p_{edge} that two vertices are joined by an edge, also known as an **Erdős-Rényi graph** [Erdős and Rényi, 1959]. Note that we are actually considering our overlay network to be an unstructured peer-to-peer network, and that we do not have any information on how it is being constructed. With a probability p_{edge} that two nodes are joined, and a total of $\binom{N}{2}$ edges, it is not difficult to see that we can expect our overlay to have $M = \frac{1}{2} \cdot p_{\text{edge}} \cdot N \cdot (N - 1)$ edges. To give an impression of what we are dealing with, Figure 4.36 shows the relationship between the number of nodes and edges for different values of p_{edge} .

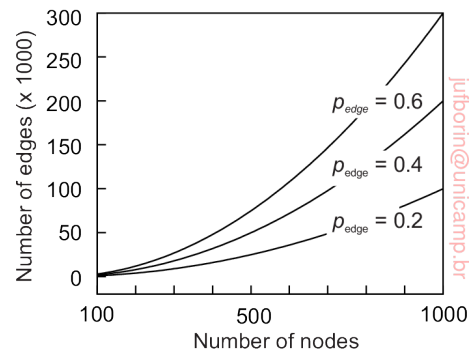


Figure 4.36: The size of a random overlay as function of the number of nodes.

To reduce the number of messages, we can also use **probabilistic flooding** as introduced by Banaei-Kashani and Shahab [2003] and formally analyzed by Oikonomou and Stavrakakis [2007]. The idea is very simple: when a node is flooding a message m and needs to forward m to a specific neighbor, it will do so with a probability p_{flood} . The effect can be dramatic: the total number of messages sent will drop linearly in p_{flood} . However, there is also a risk: the lower p_{flood} , the higher the chance that not all nodes in the network will be reached. This risk is caused by the simple fact that all neighbors of a specific node Q have decided *not* to forward m to Q . If Q has n neighbors, then this can happen roughly with a probability of $(1 - p_{flood})^n$. Clearly, the number of neighbors plays an important role in deciding whether or not to forward a message, and, indeed, we can replace the static probability of forwarding with one that takes the degree of the neighbor into account. This heuristic has been further developed and analyzed by Sereno and Gaeta [2011]. To give an idea of the efficiency of probabilistic broadcasting: in a random network of 10,000 nodes and $p_{edge} = 0.1$, we need only set $p_{flood} = 0.01$ to establish a more than 50-fold reduction in the number of messages sent in comparison to full flooding.

When dealing with a structured overlay, that is, one having a more or less deterministic topology, designing efficient flooding schemes is simpler. As an example, consider an n -dimensional hypercube, shown in Figure 4.37 for the case $n = 4$, as also discussed in Chapter 2.

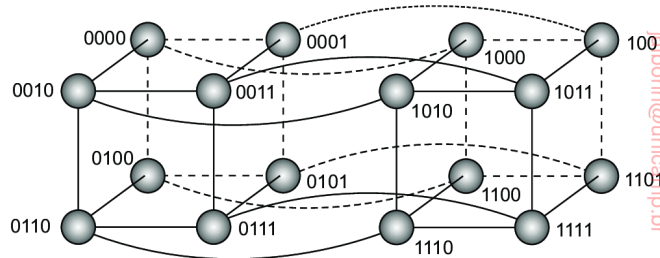


Figure 4.37: A simple peer-to-peer system organized as a four-dimensional hypercube.

A simple and efficient broadcast scheme has been designed by Schlosser et al. [2002] and relies on keeping track of neighbors *per dimension*. This is best explained by considering that every node in an n -dimensional hypercube is represented by a bit string of length n . Each edge in the overlay is labeled with its dimension. For the case $n = 4$, node 0000 will have as its neighbors the set $\{0001, 0010, 0100, 1000\}$. The edge between 0000 and 0001 is labeled “4” corresponding to changing the 4th bit when comparing 0000 to 0001 and *vice versa*. Likewise, the edge $\langle 0000, 0100 \rangle$ is labeled “2,” and so forth. A node initially broadcasts a message m to all of its neighbors, and tags m with the

label of the edge over which it sends the message. In our example, if node 1001 broadcasts a message, it will send the following:

- (m,1) to 0001
- (m,2) to 1101
- (m,3) to 1011
- (m,4) to 1000

When a node receives a broadcast message, it will forward it only along edges that have a higher dimension. In other words, in our example, node 1101 will forward m only to nodes 1111 (joined to 1101 by an edge labeled “3”) and 1100 (joined by an edge with label “4”). Using this scheme, it can be shown that every broadcast requires precisely $N - 1$ messages, where $N = 2^n$, that is the number of nodes in a n -dimensional hypercube. This broadcasting scheme is therefore optimal in terms of the number of messages sent.

Note 4.15 (Advanced: Ring-based flooding)

A hypercube is a straightforward example of how we can effectively use knowledge of the structure of an overlay network to establish efficient flooding. In the case of Chord, we can follow an approach proposed by Ghodsi [2010]. Recall that in Chord each node is identified by a number p , and each resource (typically a file), is assigned a key k from the same space as used for node identifiers. The successor $\text{succ}(k)$ of a key k is the node with the smallest identifier $p \geq k$. Consider the small Chord ring shown in Figure 4.38 and assume that node 9 wants to flood a message to all other nodes.

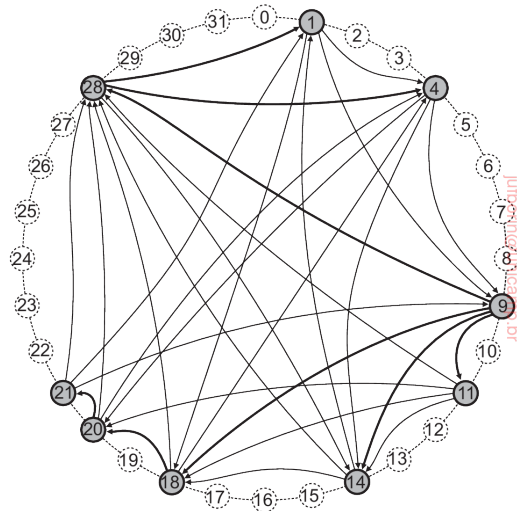


Figure 4.38: A Chord ring in which node 9 broadcasts a message.

In our example, node 9 divides the identifier space into four segments (one for each of its neighbors). Node 28 is requested to make sure that the message

reaches all nodes with identifiers $28 \leq k < 9$ (recall that we are applying modulo arithmetic); node 18 takes care of nodes with identifiers $18 \leq k < 28$; node 14 for $14 \leq k < 18$; and 11 for identifiers $11 \leq k < 14$.

Node 28 will subsequently divide the part of the identifier space it is requested to handle into two subsegments: one for its neighboring node 1 and another for 4. Likewise, node 18, responsible for segment $[18, 28)$ will “split” that segment into only one part: it has only one neighbor to delegate the flood to, and forwards the message to node 20, telling it that it should handle segment $[20, 28)$.

In the last step, only node 20 has work to do. It forwards the message to node 21, telling it to forward it to nodes known to it in the segment $[21, 28)$. As there are no such nodes anymore, the broadcast completes.

As in the case of our hypercube example, we see that flooding is done with $N - 1$ message, with N being the number of nodes in the system.

Gossip-based data dissemination

An important technique for disseminating information is to rely on **epidemic behavior**, also referred to as **gossiping**. Observing how diseases spread among people, researchers have since long investigated whether simple techniques could be developed for spreading information in very large-scale distributed systems. The main goal of these **epidemic protocols** is to rapidly propagate information among a large collection of nodes using only local information. In other words, there is no central component by which information dissemination is coordinated.

To explain the general principles of these algorithms, we assume that all updates for a specific data item are initiated at a single node. In this way, we simply avoid write-write conflicts. The following presentation is based on the classical paper by Demers et al. [1987] on epidemic algorithms. An overview of epidemic information dissemination can be found in [Eugster et al., 2004].

Information dissemination models

As the name suggests, epidemic algorithms are based on the theory of epidemics, which studies the spreading of infectious diseases. In the case of large-scale distributed systems, instead of spreading diseases, they spread information. Research on epidemics for distributed systems also aims at a completely different goal: whereas health organizations will do their utmost best to prevent infectious diseases from spreading across large groups of people, designers of epidemic algorithms for distributed systems will try to “infect” all nodes with new information as fast as possible.

Using the terminology from epidemics, a node that is part of a distributed system is called **infected** if it holds data that it is willing to spread to other nodes. A node that has not yet seen this data is called **susceptible**. Finally, an

updated node that is not willing or able to spread its data is said to have been **removed**. Note that we assume we can distinguish old from new data, for example, because it has been timestamped or versioned. In this light, nodes are also said to spread updates.

A popular propagation model is that of **anti-entropy**. In this model, a node P picks another node Q at random, and subsequently exchanges updates with Q. There are three approaches to exchanging updates:

1. P only pulls in new updates from Q
2. P only pushes its own updates to Q
3. P and Q send updates to each other (i.e., a push-pull approach)

When it comes to rapidly spreading updates, only pushing updates turns out to be a bad choice. Intuitively, this can be understood as follows. First, note that in a pure push-based approach, updates can be propagated only by infected nodes. However, if many nodes are infected, the probability of each one selecting a susceptible node is relatively small. Consequently, chances are that a particular node remains susceptible for a long period simply because it is not selected by an infected node.

In contrast, the pull-based approach works much better when many nodes are infected. In that case, spreading updates is essentially triggered by susceptible nodes. Chances are big that such a node will contact an infected one to subsequently pull in the updates and become infected as well.

If only a single node is infected, updates will rapidly spread across all nodes using either form of anti-entropy, although push-pull remains the best strategy [Jelasity et al., 2007]. Define a **round** as spanning a period in which every node will have taken the initiative once to exchange updates with a randomly chosen other node. It can then be shown that the number of rounds to propagate a single update to all nodes takes $\mathcal{O}(\log(N))$, where N is the number of nodes in the system. This indicates indeed that propagating updates is fast, but above all scalable.

Note 4.16 (Advanced: An analysis of anti-entropy)

A simple and straightforward analysis will give some idea on how well anti-entropy works. Consider a system with N nodes. One of these nodes initiates the spreading of a message m to all other nodes. Let p_i denote the probability that a node P has not yet received m after the i^{th} round. We distinguish the following three cases:

- With a pure pull-based approach, $p_{i+1} = (p_i)^2$: not only had P not yet been updated in the previous round, also the node P contacts had not yet received m .
- With a pure push-based approach, $p_{i+1} = p_i \cdot (1 - \frac{1}{N-1})^{N(1-p_i)}$: again, P should not have been updated in the previous round, but also none of the

- updated nodes should contact P. The probability that a node contacts P is $1 - \frac{1}{N-1}$; we can expect that there are $N(1 - p_i)$ updated nodes in round i .
- In a push-pull approach, we can simply combine the two: P should not contact an updated node, and should not be contacted by one.

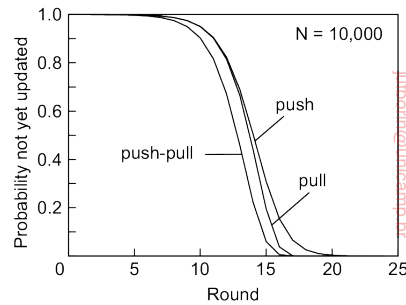


Figure 4.39: The probability of not yet having been updated as a function of the number of dissemination rounds.

Figure 4.39 shows how quickly the probability of not yet being updated drops as a function of the number of rounds. Indeed, assuming that nodes are up-and-running all the time, it turns out that anti-entropy is an extremely effective dissemination protocol.

One specific variant of epidemic protocols is called **rumor spreading**. It works as follows. If node P has just been updated for data item x , it contacts an arbitrary other node Q and tries to push the update to Q. However, it is possible that Q was already updated by another node. In that case, P may lose interest in spreading the update any further, say with probability p_{stop} . In other words, it then becomes removed.

Rumor spreading is gossiping analogous to real life. When Bob has some hot news to spread around, he may phone his friend Alice telling her all about it. Alice, like Bob, will be really excited to spread the rumor to her friends as well. However, she will become disappointed when phoning a friend, say Chuck, only to hear that the news has already reached him. Chances are that she will stop phoning other friends, for what good is it if they already know?

Rumor spreading turns out to be an excellent way of rapidly spreading news. However, it cannot guarantee that all nodes will actually be updated [Demers et al., 1987]. In fact, when there is a large number of nodes that participate in the epidemics, the fraction s of nodes that will remain ignorant of an update, that is, remain susceptible, satisfies the equation:

$$s = e^{-(1/p_{stop}+1)(1-s)}$$

Note 4.17 (Advanced: Analysis of rumor spreading)

In order to formally analyze the situation for rumor spreading, we let s denote the fraction of nodes that have not yet been updated, i.e., the fraction of susceptible nodes. Likewise, let i denote the fraction of infected nodes: the ones that have been updated and are still contacting other nodes in order to spread the news. Finally, r is the fraction of nodes that have been updated, but have given up, i.e., they are no longer passive and play no more part in disseminating news. Obviously, $s + i + r = 1$. Using theory from epidemics, it is not difficult to see the following:

$$\begin{aligned}
 (1) \quad ds/dt &= -s \cdot i \\
 (2) \quad di/dt &= s \cdot i - p_{stop} \cdot (1-s) \cdot i \\
 \Rightarrow di/ds &= -(1 + p_{stop}) + \frac{p_{stop}}{s} \\
 \Rightarrow i(s) &= -(1 + p_{stop}) \cdot s + p_{stop} \cdot \ln(s) + C
 \end{aligned}$$

where we use the notation $i(s)$ to express i as a function of s . When $s = 1$, no nodes have yet been infected, meaning that $i(1) = 0$. This allows us to derive that $C = 1 + p_{stop}$, and thus

$$i(s) = (1 + p_{stop}) \cdot (1 - s) + p_{stop} \cdot \ln(s)$$

We are looking for the situation that there is no more rumor spreading, i.e., when $i(s) = 0$. Having a closed expression for $i(s)$ then leads to

$$s = e^{-(1/p_{stop}+1)(1-s)}$$

To get an idea of what this means, take a look at Figure 4.40, which shows s as a function of p_{stop} . Even for high values of p_{stop} we see that the fraction of nodes that remains ignorant is relatively low, and always less than approximately 0.2. For $p_{stop} = 0.20$ it can be shown that $s = 0.0025$. However, in those cases when p_{stop} is relatively high, additional measures will need to be taken to ensure that *all* nodes are updated.

One of the main advantages of epidemic algorithms is their scalability, due to the fact that the number of synchronizations between processes is relatively small compared to other propagation methods. For wide-area systems, Lin and Marzullo [1999] have shown that it makes sense to take the actual network topology into account to achieve better results. In that case, nodes that are connected to only a few other nodes are contacted with a relatively high probability. The underlying assumption is that such nodes form a bridge to other remote parts of the network; therefore, they should be contacted as soon as possible. This approach is referred to as **directional gossiping** and comes in different variants.

This problem touches upon an important assumption that most epidemic solutions make, namely that a node can randomly select any other node to gossip with. This implies that, in principle, the complete set of nodes should

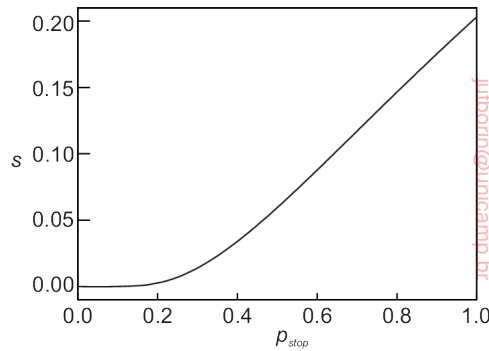


Figure 4.40: The relation between the fraction s of update-ignorant nodes and the probability p_{stop} that a node will stop gossiping once it contacts a node that has already been updated.

be known to each member. In a large system, this assumption can never hold and special measures will need to be taken to mimic such properties. We return to this issue in Section 6.7 when we discuss a peer-sampling service.

Removing data

Epidemic algorithms are extremely good for spreading updates. However, they have a rather strange side-effect: spreading the *deletion* of a data item is hard. The essence of the problem lies in the fact that deletion of a data item destroys all information on that item. Consequently, when a data item is simply removed from a node, that node will eventually receive old copies of the data item and interpret those as updates on something it did not have before.

The trick is to record the deletion of a data item as just another update, and keep a record of that deletion. In this way, old copies will not be interpreted as something new, but merely treated as versions that have been updated by a delete operation. The recording of a deletion is done by spreading **death certificates**.

Of course, the problem with death certificates is that they should eventually be cleaned up, or otherwise each node will gradually build a huge local database of historical information on deleted data items that is otherwise not used. Demers et al. [1987] propose to use what are called dormant death certificates. Each death certificate is timestamped when it is created. If it can be assumed that updates propagate to all nodes within a known finite time, then death certificates can be removed after this maximum propagation time has elapsed.

However, to provide hard guarantees that deletions are indeed spread to all nodes, only a very few nodes maintain dormant death certificates that are

never thrown away. Assume node P has such a certificate for data item x . If by any chance an obsolete update for x reaches P , P will react by simply spreading the death certificate for x again.

4.5 Summary

Having powerful and flexible facilities for communication between processes is essential for any distributed system. In traditional network applications, communication is often based on the low-level message-passing primitives offered by the transport layer. An important issue in middleware systems is to offer a higher level of abstraction that will make it easier to express communication between processes than the support offered by the interface to the transport layer.

One of the most widely used abstractions is the Remote Procedure Call (RPC). The essence of an RPC is that a service is implemented by means of a procedure, of which the body is executed at a server. The client is offered only the signature of the procedure, that is, the procedure's name along with its parameters. When the client calls the procedure, the client-side implementation, called a stub, takes care of wrapping the parameter values into a message and sending that to the server. The latter calls the actual procedure and returns the results, again in a message. The client's stub extracts the result values from the return message and passes it back to the calling client application.

RPCs offer synchronous communication facilities, by which a client is blocked until the server has sent a reply. Although variations of either mechanism exist by which this strict synchronous model is relaxed, it turns out that general-purpose, high-level message-oriented models are often more convenient.

In message-oriented models, the issues are whether or not communication is persistent, and whether or not communication is synchronous. The essence of persistent communication is that a message that is submitted for transmission, is stored by the communication system as long as it takes to deliver it. In other words, neither the sender nor the receiver needs to be up and running for message transmission to take place. In transient communication, no storage facilities are offered, so that the receiver must be prepared to accept the message when it is sent.

In asynchronous communication, the sender is allowed to continue immediately after the message has been submitted for transmission, possibly before it has even been sent. In synchronous communication, the sender is blocked at least until a message has been received. Alternatively, the sender may be blocked until message delivery has taken place or even until the receiver has responded as with RPCs.

Message-oriented middleware models generally offer persistent asynchronous communication, and are used where RPCs are not appropriate. They are often used to assist the integration of (widely dispersed) collections of databases into large-scale information systems.

Finally, an important class of communication protocols in distributed systems is multicasting. The basic idea is to disseminate information from one sender to multiple receivers. We have discussed two different approaches. First, multicasting can be achieved by setting up a tree from the sender to the receivers. Considering that it is now well understood how nodes can self-organize into peer-to-peer system, solutions have also appeared to dynamically set up trees in a decentralized fashion. Second, flooding messages across the network is extremely robust, yet requires special attention if we want to avoid severe waste of resources as nodes may see messages multiple times. Probabilistic flooding by which a node forwards a message with a certain probability often proves to combine simplicity and efficiency, while being highly effective.

Another important class of dissemination solutions deploys epidemic protocols. These protocols have proven to be very simple and extremely robust. Apart from merely spreading messages, epidemic protocols can also be efficiently deployed for aggregating information across a large distributed system.